Data structures and algorithms

Instructor: Masoud Asghari

Web page: http://www.maser.ir/lectures/dsas2017sut/

Ch: 0.1 (**Preliminary C++**)

In this Slide Series:

- C++ Mainly from Ref.1
  - Preliminaries (Reminders)
    - Structure of a program
    - Variables and types
    - Constants
    - Operators
    - Basic Input/Output
    - Control Structures
    - Functions
    - Arrays
    - Character sequences

Compilers

- Computers understand only language consists of sets of instructions made of ones and zeros.

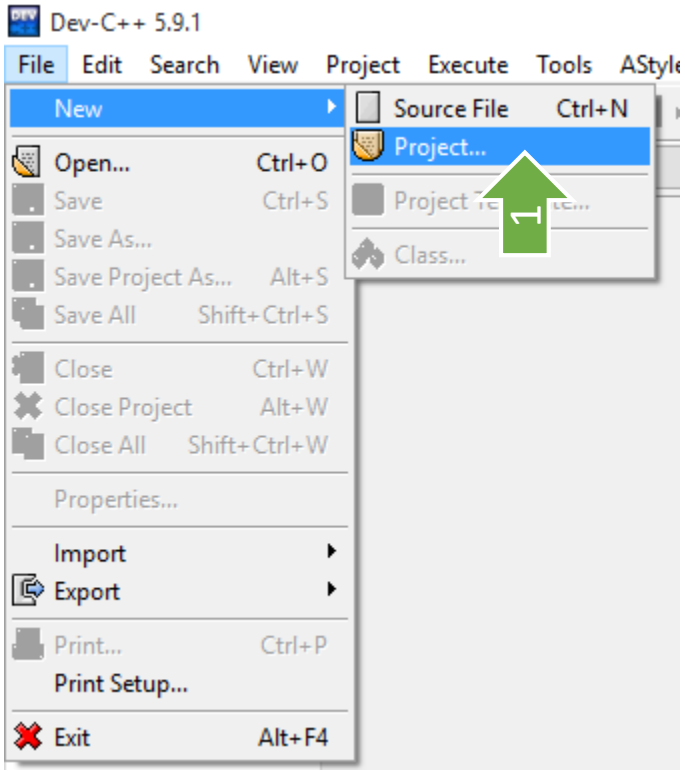
- All Codes at Programming languages have to be re-written (translated) into machine language at some point

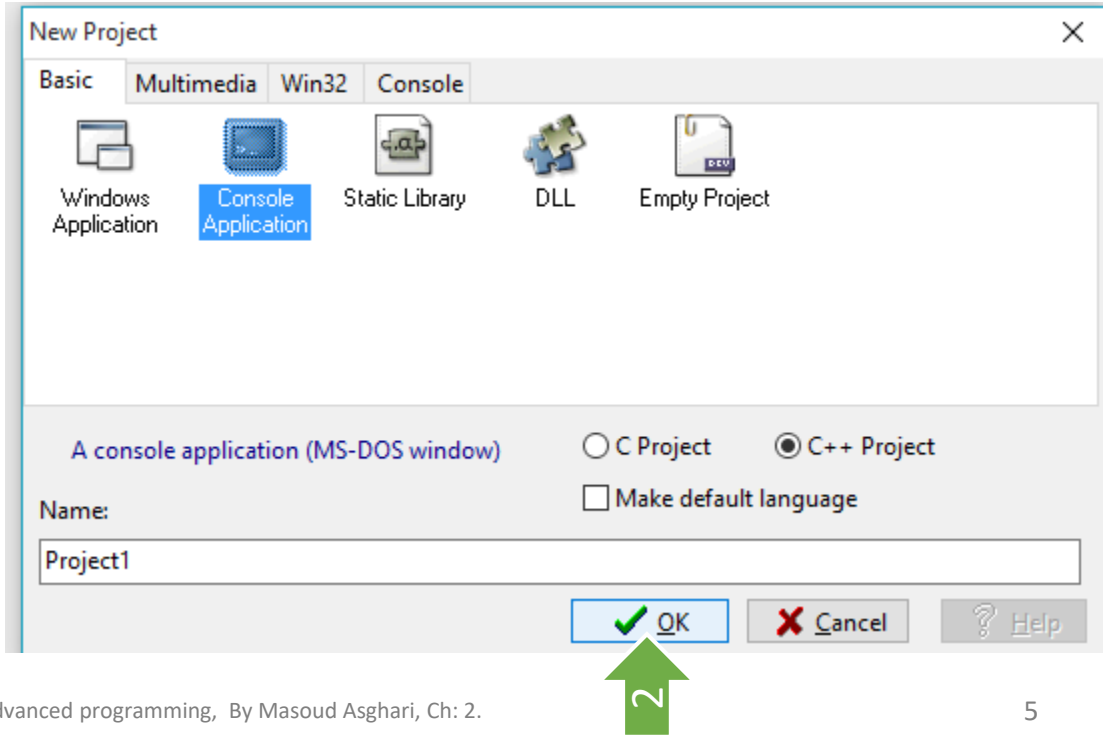
- This is done by special programs called **compilers** and linkers

Compilers

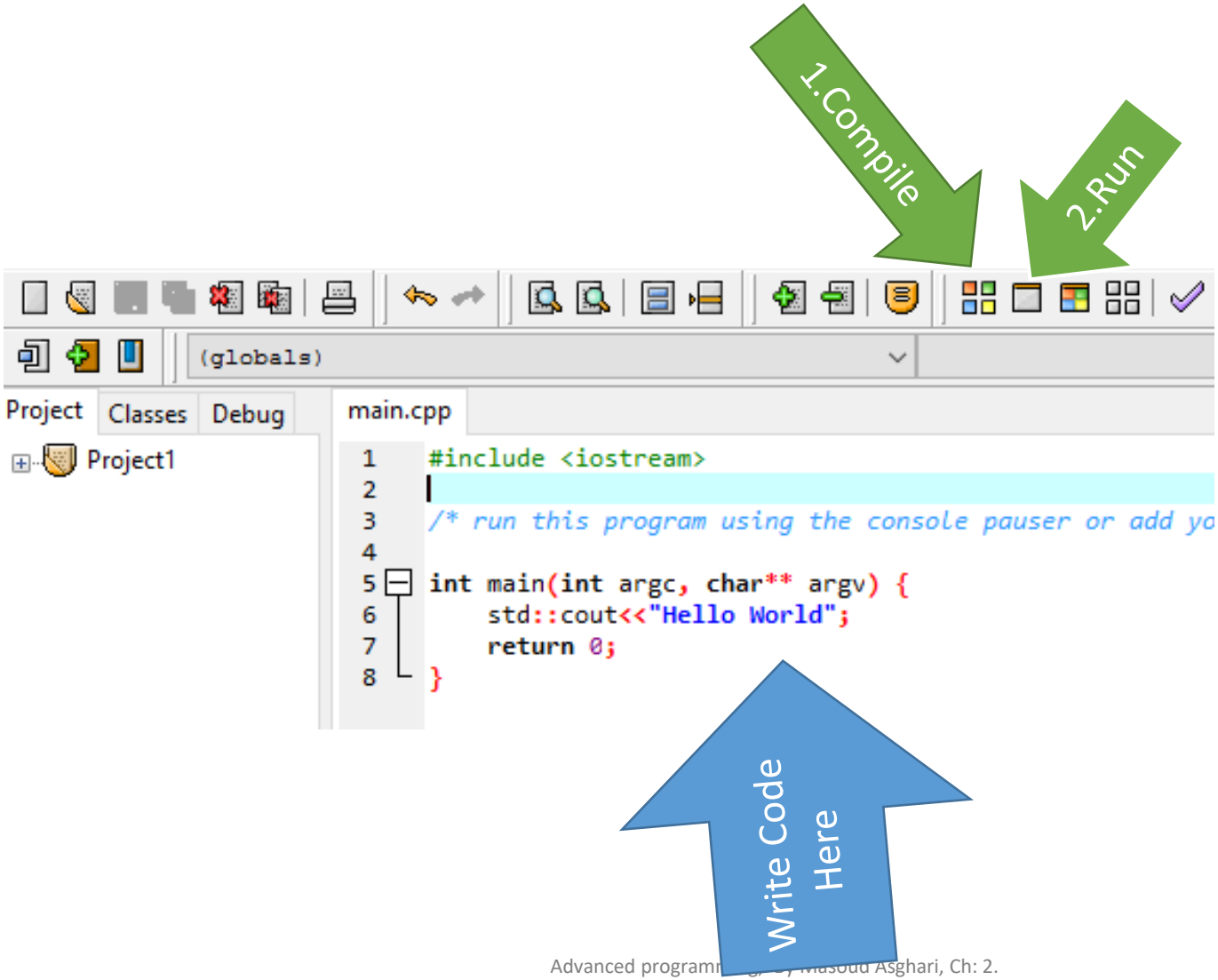
- You Write codes in C++ Language
- The code that you wrote should be converted to machine code
  - Compiler does this, example: **GCC**
- You can write your codes in note pad and then compile manually
  - But it is hard to use
  - That is why we use IDEs
- **Integrated development environment** (**IDE**)
  - source code editor
  - build automation tools (Including Compiler)
  - Debugger
  - intelligent code completion
- We use Dev C++ as an IDE with compiler
  - Download and install it from: http://www.bloodshed.net/dev/devcpp.html
  - ***Dev-C++ 5.0 beta 9.2 (4.9.9.2) (*9.0 MB) with Mingw/GCC 3.4.2**

Getting started with Dev C++





Getting started with Dev C++



Getting started with Dev C++

- Results:
  - Is an compiled Exe file with proper machine codes

```
C:\Users\mas\Documents\c_projects\Project1.exe
Hello World
--------------------------------
Process exited after 0.006821 seconds with return value 0
Press any key to continue . . .
```

Console programs

- Console programs
  - are programs that use text
  - to communicate with the user and the environment
  - such as printing text to the screen or reading input from a keyboard.

```
C:\Users\mas\Documents\c_projects\Project1.exe
Hello World
--------------------------------
Process exited after 0.006821 seconds with return value 0
Press any key to continue . . .
```

- GUI Programs
  - Programs with graphical interfaces (Windows)

Structure of a program

- In C++, the separation between statements is specified with an ending semicolon (;)
  - Syntax:

```
int i=2; int j=3; int x; x= i +j;
std::cout<<x;
```

- Comments:
  - To document directly within the source code
  - Give information about your codes
  - comments do not affect the operation of the program
  - Syntax:

```
int i=2; // line comment = until end of this line
int i=2; /* block comment */ int j=3;
```

Using Name spaces

- all the elements in the standard C++ library are declared within what is a called a namespace:
  - the namespace std
- As seen on

```
std::cout<<"Hello World";
```

  - Called *explicit qualification*

- You can refer to a namespace's things (functions) without using *its_name::* by declaring the name space at the beginning of your codes

```
#include <iostream>
using namespace std; // declaring std here

int main(int argc, char** argv) {
    cout<<"Hello World"; //Did not use std::
    return 0;
}
```

Variables and types

- *variable* is a portion of memory to store a value.
- Each variable needs

```
int test_1=100; double test_2=1.75;
```

  - a name that identifies it, called identifier
  - a type that tells the kind of variable to the compiler
  - Assigning a value to the variable
- A valid identifier
  - Is a sequence of one or more letters, digits, or underscore characters (_).
  - Spaces, punctuation marks, and symbols cannot be part of an identifier.
  - Identifiers shall always begin with a letter.
  - Identifiers are Case sensitive

```
int i=1; int I=2; //i is not equal with I
```

Variables and types

- The values of variables are stored somewhere in an unspecified location in the computer memory as zeros and ones.
- We can simply refer to it by its name.
- Different Type of variables
  - are not interpreted in the same way,
  - don't occupy the same amount of memory.
- That is why we need to declare the type of a variable when first defining it.
- Fundamental data types
  - are basic types implemented directly by the language.
- You can define your personal types (Classes, struct, enum) and declare variables of these types too.

fundamental types in C++

Group	Type names*	Notes on size / precision
Character	**char**	1byte
Boolean type	**bool**	true, false
Integer	**short**	2bytes -32768 to 32767
	int	4bytes -2147483648 to 2147483647
Floating-point types	**float**	4bytes (~7 digits)
	double	8bytes (~15 digits)
Void type	**void**	no storage

All types are available at: http://www.tutorialspoint.com/cplusplus/cpp_data_types.htm

Declaration of variables

- C++ requires every variable to be declared with its type before its first use.
  - This informs the compiler the size to reserve in memory for the variable and how to interpret its value
- The syntax to declare a new variable in C++
  - write the type followed by the variable name

```
int a;
float mynumber;
```

- You can declare more than one variable at the same time:

```
int a, b, c;
```

- At the time of declaration, you can initialize the variable by
  - appending an equal sign followed by the value to which the variable is initialized

```
float mynumber = 7.51;
```

Constants

- *Constants* are expressions with a fixed value.
  - **Integer Numerals:** 256421
  - **Floating Point Numerals:** 25.241
  - **Character and string literals**

```
'p'
"Hello world"
```

  - **single character escape codes:**

Escape code	Description
\n	newline
\r	carriage return
\t	tab
\'	single quote (')
\"	double quote (")
\?	question mark (?)
\\	backslash (\)

  - **true, false, nullptr**
- User defined constants
  - Add const before type

```
const double pi = 3.1415926;
const char tab = '\t';
```

- Preprocessor definitions
  - **#define identifier replacement**

```
#define PI 3.14159
```

  - No semicolon is used after Preprocessor codes (those lines start with #)

Operators

- **Assignment operator (=)**
  - assigns a value to a variable
  - always takes place from right to left
  - Type of the assignment should be matched
    - May need to explicit type casting to prevent compiler errors
  - Multiple assignment is possible

```
int a, b;          // a:?,  b:?
a = 10;            // a:10, b:?
b = 4;             // a:10, b:4
a = b;             // a:4,  b:4
b = 7;             // a:4,  b:7
```

```
x = y = z = 5;
```

- **Arithmetic operators**

operator	description
+	addition
-	subtraction
*	multiplication
/	division
%	modulo

```
int x;
x= 10 %3; //x equals 1
```

- **Compound assignment**

expression	equivalent to...
y += x;	y = y + x;
x -= 5;	x = x - 5;
x /= y;	x = x / y;
price *= units + 1;	price = price * (units+1);

Operators

- **Increment and decrement**
  - increase operator (++) and decrease operator (--)
    increase or reduce by one the value stored in a variable
  - it can be used both as a prefix and as a suffix
    - In both ways (++x) and (x++), the value is increased.
    - In case of a prefix (++x), the expression evaluates to the final value of x, once it is already increased.
    - In case of a suffix (x++), the expression evaluates to the value that x had before being increased.

```
++x;     //all equivalent
x+=1;    //all equivalent
x=x+1;   //all equivalent
```

Example 1	Example 2

`x = 3;`
`y = ++x;`
`// x contains 4, y contains 4`
	`x = 3;`
	`y = x++;`
	`// x contains 4, y contains 3`

Operators

- **Relational and comparison operators**
  - Two expressions can be compared using relational and equality operators.
  - For example, to know if two values are equal or if one is greater than the other
  - The result of such an operation is either true or false (i.e., a Boolean value).

operator	description
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

```
(7 == 5)     // evaluates to false
(5 > 4)      // evaluates to true
(3 != 2)     // evaluates to true
(6 >= 6)     // evaluates to true
(5 < 5)      // evaluates to false
```

```
int a=2, b=3, c=6;
(a == 5);     // evaluates to false, since a is not equal to 5
(a*b >= c);   // evaluates to true, since (2*3 >= 6) is true
(b+4 > a*c);  // evaluates to false, since (3+4 > 2*6) is false
((b=2) == a); // evaluates to true
```

- Be careful! The assignment operator (operator =, with one equal sign) is not the same as the equality comparison operator (operator ==, with two equal signs);

Operators

- **Logical operators**
  - The operator ! is the C++ operator for the Boolean operation NOT.
    - It has only one operand, to its right, and inverts it

```
!(5 == 5)   // evaluates to false because the expression at its right (5 == 5) is true
!(6 <= 4)   // evaluates to true because (6 <= 4) would be false
!true       // evaluates to false
!false      // evaluates to true
```

  - The logical operators && and || are used when evaluating two expressions to obtain a single relational result.
  - The operator && corresponds to Boolean logical operation AND, yields true if both its operands are true, and false otherwise
  - The operator || corresponds to Boolean logical operation OR, yields true if either of its operands is true, thus being false only when both operands are false.

&& OPERATOR (and)		
a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

|| OPERATOR (or)		
a	b	a || b
true	true	true
true	false	true
false	true	true
false	false	false

```
( (5 == 5) && (3 > 6) )  // evaluates to false ( true && false )
( (5 == 5) || (3 > 6) )  // evaluates to true ( true || false )
```

Operators

- **Conditional ternary operator**
  - *condition ? result1 : result2 ;*
  - The conditional operator evaluates an expression,
  - returning one value if that expression evaluates to true,
  - and a different one if the expression evaluates as false.

```
int a,b,c;
a=2;
b=7;
c = (a>b) ? a : b;
cout << c << '\n'; //prints 7
```

- **Bitwise operators**
  - Bitwise operators modify variables considering the bit patterns that represent the values they store.

operator	asm equivalent	description
&	AND	Bitwise AND
|	OR	Bitwise inclusive OR
^	XOR	Bitwise exclusive OR
~	NOT	Unary complement (bit inversion)
<<	SHL	Shift bits left
>>	SHR	Shift bits right

Operators

- **Explicit type casting operator**
  - Type casting operators allow to convert a value of a given type to another type.
  - One way: precede the expression to be converted by the new type enclosed between parentheses ()

```
int i;
float f = 3.14;
i = (int) f;
```

  - Another way : use the functional notation preceding the expression to be converted by the type and enclosing the expression between parentheses

```
i = int (f);
```

- **sizeof**
  - accepts one parameter, which can be either a type or a variable, and returns the size in bytes of that type or object

```
x = sizeof (char);
```

Precedence of operators

```
x = 5 + 7 % 2;      // x = 6
x = 5 + (7 % 2);    // x = 6 (same as without parenthesis)
x = (5 + 7) % 2;    // x = 0
```

From greatest to smallest priority, C++ operators are evaluated in the following order:

Level	Precedence group	Operator	Description	Grouping
1	Scope	`::`	scope qualifier	Left-to-right
2	Postfix (unary)	`++ --`	postfix increment / decrement	Left-to-right
		`()`	functional forms	
		`[]`	subscript	
		`. ->`	member access	
3	Prefix (unary)	`++ --`	prefix increment / decrement	Right-to-left
		`~ !`	bitwise NOT / logical NOT	
		`+ -`	unary prefix	
		`& *`	reference / dereference	
		`new delete`	allocation / deallocation	
		`sizeof`	parameter pack	
		`(type)`	C-style type-casting	
4	Pointer-to-member	`.* ->*`	access pointer	Left-to-right
5	Arithmetic: scaling	`* / %`	multiply, divide, modulo	Left-to-right
6	Arithmetic: addition	`+ -`	addition, subtraction	Left-to-right
7	Bitwise shift	`<< >>`	shift left, shift right	Left-to-right
8	Relational	`< > <= >=`	comparison operators	Left-to-right
9	Equality	`== !=`	equality / inequality	Left-to-right
10	And	`&`	bitwise AND	Left-to-right
11	Exclusive or	`^`	bitwise XOR	Left-to-right
12	Inclusive or	`|`	bitwise OR	Left-to-right
13	Conjunction	`&&`	logical AND	Left-to-right
14	Disjunction	`||`	logical OR	Left-to-right
15	Assignment-level expressions	`= *= /= %= += -= >>= <<= &= ^= |=`	assignment / compound assignment	Right-to-left
		`?:`	conditional operator	
16	Sequencing	`,`	comma separator	Left-to-right

Basic Input/Output

- C++ uses a convenient abstraction called *streams* to perform input and output operations in sequential media such as the screen, the keyboard or a file
  - A *stream* is an entity where a program can either insert or extract characters to/from

stream	description
cin	standard input stream
cout	standard output stream

- **Standard output**
  - cout is used together with the insertion operator, that is <<
  - << operator inserts the data that follows it into the stream that precedes it.

```
cout << "Output sentence"; // prints Output sentence on screen
cout << 120;              // prints number 120 on screen
cout << x;                // prints the value of x on screen
```

  - Multiple insertion operations (<<) may be chained in a single statement:

```
cout << "This " << " is a " << "single C++ statement";
```

```
cout << "I am " << age << " years old and my zipcode is " << zipcode;
```

Basic Input/Output

- To insert a line break,
  - a new-line character shall be inserted at the exact position the line should be broken.
  - a new-line character can be specified as \n

```
cout << "First sentence.\n";
cout << "Second sentence.\nThird sentence.";
```

This produces the following output:

```
First sentence.
Second sentence.
Third sentence.
```

- **Standard input**
  - cin is used together with the extraction operator, which is >>
  - It then followed by the variable where the extracted data is stored

```
int i;
cout << "Please enter an integer value: ";
cin >> i;
cout << "The value you entered is " << i;
cout << " and its double is " << i*2 << ".\n";
```

```
Please enter an integer value: 702
The value you entered is 702 and its double is 1404.
```

Flow control

- Programs are not limited to a linear sequence of statements
- a program may repeat segments of code, or take decisions
- **Selection statements**
  - *if (condition) statement of true; else statement of flase;*
    - condition is the expression that is being evaluated = 0 (false), not zero (true)
    - Statement is the code that we want to execute, including:
      - A single statement with ; at the end.
      - Group of statements enclosed in { }
        - No need for ; after { }
    - *else statement of flase;* is optional
    - Chaining multiple conditions by:
      - *else if (condition) statement of true;*

```
if (x == 100)
{
   cout << "x is ";
   cout << x;
}
```

```
int x;
if (x == 0) cout << "x is positive";
else if (x < 0) cout << "x is negative";
else if (false) cout << "never happens";
else { cout << "x is 0"; x=1; }
```

Flow control

- **Selection statements**
  - Its purpose is to check for a value among a number of possible **constant** expressions

```
switch (expression)
{
  case constant1:
     group-of-statements-1;
     break;
  case constant2:
     group-of-statements-2;
     break;
  .
  .
  .
  default:
     default-group-of-statements
}
```

switch example	if-else equivalent

`switch (x) {`
`  case 1:`
`  cout << "x is 1";`
`  break;`
`  case 2:`
`  cout << "x is 2";`
`  break;`
`  default:`
`  cout << "value of x unknown";`
`  }`
	`if (x == 1) {`
	`  cout << "x is 1";`
	`}`
	`else if (x == 2) {`
	`  cout << "x is 2";`
	`}`
	`else {`
	`  cout << "value of x unknown";`
	`}`

  - break; is needed at the end of each case constant:
  - default: is optional , if all cases does not match, then default is executed
    - No need for break; at default section.
  - More than one statements without grouping with { } can be written at each case

Flow control

```
switch (expression)
{
  case constant1:
     group-of-statements-1;
     break;
  case constant2:
     group-of-statements-2;
     break;
  .
  .
  .
  default:
     default-group-of-statements
}
```

- If break; is not included, all statements following the case (including those under any other labels) are also executed, until the end of the switch block or a break; is reached.

```
switch (x) {
  case 1:
  case 2:
  case 3:
    cout << "x is 1, 2 or 3";
    break;
  default:
    cout << "x is not 1, 2 nor 3";
  }
```

Flow control - loops

- Loops repeat a statement a certain number of times, or while a condition is fulfilled.
- **The while loop** `while (expression) statement`
  - repeats statement while expression is true. If, after any execution of statement, expression is no longer true, the loop ends, and the program continues right after the loop.
  - Use single statement with ; or use group of statements enclosed within { }

```
// custom countdown using while
#include <iostream>
using namespace std;

int main ()
{
  int n = 10;

  while (n>0) {
    cout << n << ", ";
    --n;
  }

  cout << "liftoff!\n";
}
```

```
int n = 10;

while (true) {
    cout << n << ", ";
    n--;
    if(n<=0)break;
}

cout << "liftoff!\n";
```

```
10, 9, 8, 7, 6, 5, 4, 3, 2, 1, liftoff!
```

- **The do-while loop**

```
do statement while (condition);
```

  - It behaves like a while-loop, except that condition is evaluated after the execution of statement instead of before, guaranteeing at least one execution of statement, even if condition is never fulfilled.

- **The for loop**

```
for (initialization; condition; increase) statement;
```

  - The for loop is designed to iterate a number of times.
  - initialization expression executed before the loop begins the first time
    - It is executed for only a single time
    - You can declare variables there and initialize it too
  - condition is checked before each iteration
    - It should return true or false as the condition evaluated
  - increase expression executed after each iteration
  - Use single statement with ; or use group of statements enclosed within { }
  - Any of initialization , condition , increase can be blank

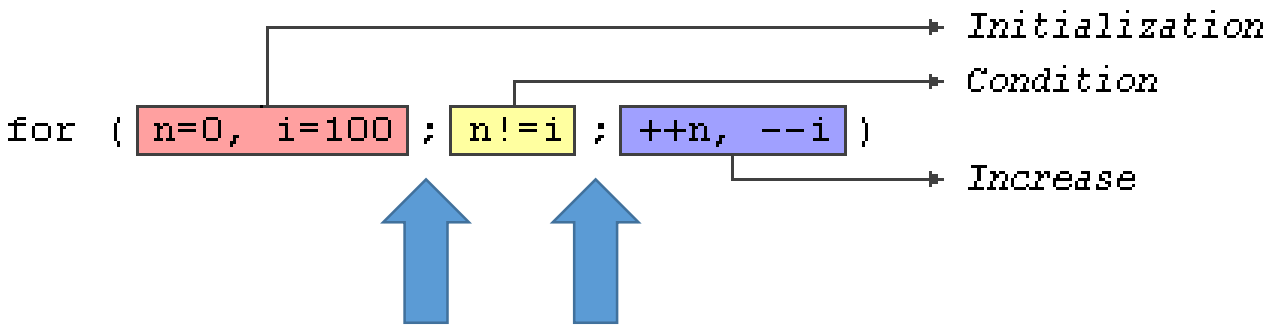
```
using namespace std;

int main ()
{
  for (int n=10; n>0; n--) {
    cout << n << ", ";
  }
  cout << "liftoff!\n";
}
```

```
int n = 10;
for ( ; ; ) {
    cout << n << ", ";
    n--;
    if(n<=0)break;
}
cout << "liftoff!\n";
```

- **The for loop**
  - it may be useful to execute more than a single expression as any of *initialization*, *condition*, or *statement*.

```
for ( n=0, i=100 ; n!=i ; ++n, --i )
{
   // whatever here...
}
```

This loop will execute 50 times if neither `n` or `i` are modified within the loop:



Flow control - loops

- **The break statement**
  - break; leaves a loop,
    - even if the condition for its end is not fulfilled.
  - It can be used to end an infinite loop,
    - or to force it to end before its natural end.

```
int n = 10;
for ( ; ; ) {
    cout << n << ", ";
    n--;
    if(n<=0)break;
}
cout << "liftoff!\n";
```

- **The continue statement**
  - continue; causes the program to skip the rest of the loop in the current iteration
    - as if the end of the statement block had been reached,
    - causing it to jump to the start of the following iteration.

```
// continue loop example
#include <iostream>
using namespace std;

int main ()
{
  for (int n=10; n>0; n--) {
    if (n==5) continue;
    cout << n << ", ";
  }
  cout << "liftoff!\n";
}
```

```
10, 9, 8, 7, 6, 4, 3, 2, 1, liftoff!
```

Functions

- Functions allow to structure programs in segments of code to perform individual tasks.
- It Prevents writing the same code for multiple times
- It Allows recursive algorithms

- It is similar to mathematical function
  - It can get multiple arguments (parameters) or even no parameter
  - And return a single unique value depending on its calling parameters

- You can reuse your functions in your other projects

- Function is a block of codes which returns a result with the given parameters
  - It only encloses codes
  - It does not enclose data
  - struct in the other hand only encloses data
  - Objects (classes) in contrast enclose <u>both</u> codes and data

Functions

- In C++, a function is a group of statements enclosed in { } that is given a name, and which can be called from some point of the program.

```
type name ( parameter1, parameter2, ...) { statements }
```

  - - type is the type of the value returned by the function.
  - - name is the identifier by which the function can be called.
  - - parameters (as many as needed): Each parameter consists of a type followed by an identifier, with each parameter being separated from the next by a comma.
  - Each parameter looks very much like a regular variable declaration (example: int x), and in fact acts within the function as a regular variable which is local to the function. The purpose of parameters is to allow passing arguments to the function from the location where it is called from.
  - All parameters and variables declared inside a function are local to that function
    - You can not see and access them outside of function
  - - statements is the function's body. It is a block of statements surrounded by braces { } that specify what the function actually does.
  - When return value; is called inside the statement,
    - the function ends, and returns the control back to the point where the function was called.
    - The return value should have the type of the declared function

Functions

- Example:

```
// function example
#include <iostream>
using namespace std;

int addition (int a, int b)
{
  int r;
  r=a+b;
  return r;
}

int main ()
{
  int z;
  z = addition (5,3);
  cout << "The result is " << z;
}
```

```
The result is 8
```

- Program always starts by calling main.
- main is the only function called automatically, and the code in any other function is only executed if its function is called from main (directly or indirectly).

Functions

- A function can actually be called multiple times within a program

```
// function example
#include <iostream>
using namespace std;

int subtraction (int a, int b)
{
  int r;
  r=a-b;
  return r;
}

int main ()
{
  int x=5, y=3, z;
  z = subtraction (7,2);
  cout << "The first result is " << z << '\n';
  cout << "The second result is " << subtraction (7,2) << '\n';
  cout << "The third result is " << subtraction (x,y) << '\n';
  z= 4 + subtraction (x,y);
  cout << "The fourth result is " << z << '\n';
}
```

```
The first result is 5
The second result is 5
The third result is 2
The fourth result is 6
```

Functions with no type. The use of void

- if the function does not need to return a value, the type to be used is void

```
// void function example
#include <iostream>
using namespace std;

void printmessage ()
{
  cout << "I'm a function!";
}

int main ()
{
  printmessage ();
}
```

```
I'm a function!
```

- All functions with a return type shall end with a proper return statement that includes a return value, even if this is never used.

Functions

- **Arguments passed by value and by reference**
  - Will be discussed In pointer section
- **Efficiency considerations and const references**
  - Will be discussed In pointer section

Functions

- **Default values in parameters**
  - functions can also have optional parameters,
  - for which no arguments are required in the call, in such a way that, for example, a function with three parameters may be called with only two.
  - For this, the function shall include a default value for its last parameter, which is used by the function when called with fewer arguments

```
// default values in functions
#include <iostream>
using namespace std;

int divide (int a, int b=2)
{
  int r;
  r=a/b;
  return (r);
}

int main ()
{
  cout << divide (12) << '\n';
  cout << divide (20,4) << '\n';
  return 0;
}
```

```
6
5
```

- **Declaring functions**
  - In C++, identifiers can only be used in expressions once they have been declared.
  - Functions cannot be called before they are declared.
  - That is why, the functions were always defined before the main function, which is the function from where the other functions were called.
  - What if you have two functions calling each other
  - To solve this issue
    - The prototype of a function can be declared without actually defining the function completely
    - giving just enough details to allow the types involved in a function call to be known by compiler
  - prototype declaration include all types involved
    - the return type and the type of its arguments
      - using the same syntax as used in the definition of the function
    - but replacing the body of the function with an semicolon ;
    - The parameter list does not need to include the parameter names, but only their types

```
int protofunction (int first, int second);
int protofunction (int, int);
```

Function prototype example

```
1  // declaring functions prototypes
2  #include <iostream>
3  using namespace std;
4
5  void odd (int x);
6  void even (int x);
7
8  int main()
9  {
10   int i;
11   do {
12     cout << "Please, enter number (0 to exit): ";
13     cin >> i;
14     odd (i);
15   } while (i!=0);
16   return 0;
17 }
18
19 void odd (int x)
20 {
21   if ((x%2)!=0) cout << "It is odd.\n";
22   else even (x);
23 }
24
25 void even (int x)
26 {
27   if ((x%2)==0) cout << "It is even.\n";
28   else odd (x);
29 }
```

```
Please, enter number (0 to exit): 9
It is odd.
Please, enter number (0 to exit): 6
It is even.
Please, enter number (0 to exit): 1030
It is even.
Please, enter number (0 to exit): 0
It is even.
```

Function

- **Recursively**
  - functions call themselves.
  - It is useful for some tasks, such as sorting elements, or calculating the factorial of numbers.
  - **Example: factorial of a number (*n*!)**

```
// factorial calculator
#include <iostream>
using namespace std;

long factorial (long a)
{
  if (a > 1)
   return (a * factorial (a-1));
  else
   return 1;
}

int main ()
{
  long number = 9;
  cout << number << "! = " << factorial (number);
  return 0;
}
```

```
9! = 362880
```

Function

- Overloaded functions
  - two different functions can have the same name if their parameters are different (in numbers or types)
  - Return type does not count

```
// overloading functions
#include <iostream>
using namespace std;

int operate (int a, int b)
{
  return (a*b);
}

double operate (double a, double b)
{
  return (a/b);
}

int main ()
{
  int x=5,y=2;
  double n=5.0,m=2.0;
  cout << operate (x,y) << '\n';
  cout << operate (n,m) << '\n';
  return 0;
}
```

```
10
2.5
```

Function

- **Scopes**

```
int foo;        // global variable

int some_function ()
{
  int bar;      // local variable
  bar = 0;
}

int other_function ()
{
  foo = 1;  // ok: foo is a global variable
  bar = 2;  // wrong: bar is not visible from this function
}
```

```
int some_function ()
{
  int x;
  x = 0;
  double x;   // wrong: name already used in this scope
  x = 0.0;
}
```

```
// inner block scopes
#include <iostream>
using namespace std;

int main () {
  int x = 10;
  int y = 20;
  {
    int x;   // ok, inner scope.
    x = 50;  // sets value to inner x
    y = 50;  // sets value to (outer) y
    cout << "inner block:\n";
    cout << "x: " << x << '\n';
    cout << "y: " << y << '\n';
  }
  cout << "outer block:\n";
  cout << "x: " << x << '\n';
  cout << "y: " << y << '\n';
  return 0;
}
```

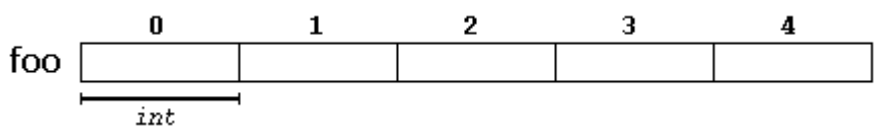
```
inner block:
x: 50
y: 50
outer block:
x: 10
y: 50
```

Arrays

- An array is a series of elements of the same type
  - placed in contiguous memory locations
  - that can be individually referenced by adding an index to a unique identifier.
- Declaration Syntax: `type name [elements];`
  - type is a valid type (such as int, float...),
  - name is a valid identifier
  - elements (which is always enclosed in square brackets []), specifies the length of the array in terms of the number of elements
    - It should be constant number
  - Example: the foo array, with five elements of type int, can be declared as:

```
int foo [5];
```

  - This allocates following spaces in the memory



  - the first element in an array is always numbered with a zero (not a one), no matter its length.

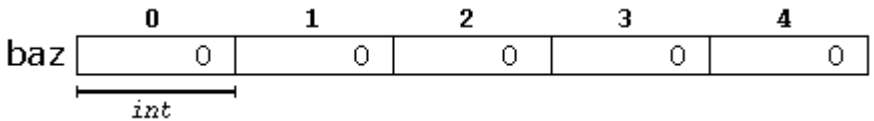
Arrays

- **Initializing arrays**
  - Elements in an array can be explicitly initialized to specific values when it is declared, by enclosing those initial values in braces { } .

```
int foo [5] = { 16, 2, 77, 40, 12071 };
```

  - If count of initial values is less than length of array
    - the remaining elements are set to their default values

```
int baz [5] = { };
```



  - When explicitly initialized is provided for an array, you can leave the square brackets empty [ ] .

```
int foo [] = { 16, 2, 77, 40, 12071 };
```

- Note: length (i.e., size) of the array cannot be changed during program execution.
  - It has constant length, defining during declaration.
  - What if we need dynamic length of elements???
  - Pointers and memory management (allocation, release) is the answer

Arrays

- **Accessing the values of an array**
  - The values of any of the elements in an array can be accessed by
  - Array_name[index]
    - Where index= 0 , 1 , … , (length – 1)

	foo[0]	foo[1]	foo[2]	foo[3]	foo[4]
foo					

- Examples:

```
foo [2] = 75;
```

```
x = foo[2];
```

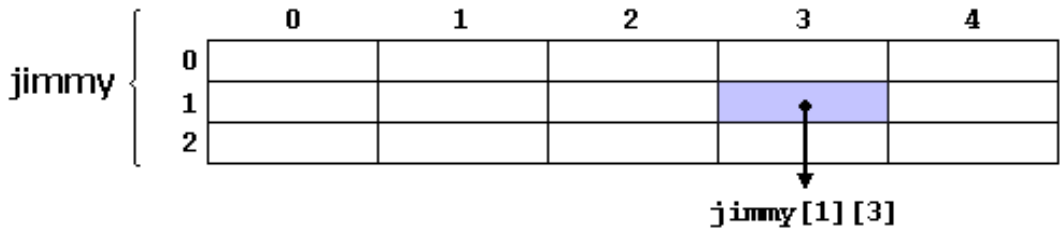
```
// arrays example
#include <iostream>
using namespace std;

int foo [] = {16, 2, 77, 40, 12071};
int n, result=0;

int main ()
{
  for ( n=0 ; n<5 ; ++n )
  {
    result += foo[n];
  }
  cout << result;
  return 0;
}
```

```
12206
```

Arrays

- **Multidimensional arrays**
  - arrays of arrays
  - Example

```
int jimmy [3][5];
```



  - Multidimensional arrays are not limited to two indices

multidimensional array	pseudo-multidimensional array

`#define WIDTH 5`
`#define HEIGHT 3`
 	
`int jimmy [HEIGHT][WIDTH];`
`int n,m;`
 	
`int main ()`
`{`
`  for (n=0; n<HEIGHT; n++)`
`  for (m=0; m<WIDTH; m++)`
`  {`
`  jimmy[n][m]=(n+1)*(m+1);`
`  }`
`}`
	`#define WIDTH 5`
	`#define HEIGHT 3`
	
	`int jimmy [HEIGHT * WIDTH];`
	`int n,m;`
	
	`int main ()`
	`{`
	`  for (n=0; n<HEIGHT; n++)`
	`  for (m=0; m<WIDTH; m++)`
	`  {`
	`  jimmy[n*WIDTH+m]=(n+1)*(m+1);`
	`  }`
	`}`

jimmy

	0	1	2	3	4
0	1	2	3	4	5
1	2	4	6	8	10
2	3	6	9	12	15

Arrays

- **Arrays as parameters**
  - we may need to pass an array to a function as a parameter.
  - Here, array is passed as reference
    - Only address of the array is passed to the function
    - Means no new copy is made and changes inside function affect the actual array
  - To accept an array as parameter for a function,
    - the parameters can be declared as the array type,
    - but with empty brackets [ ], omitting the actual size of the array.

```
void procedure (int arg[])
```

  - In order to pass to this function an array `int myarray [40];`
    - it would be enough to write a call like this: `procedure (myarray);`

  - Note: arrays cannot be directly copied, and thus what is really passed is a pointer.

Arrays

- **Arrays as parameters**
  - **Example (full code)**

```cpp
// arrays as parameters
#include <iostream>
using namespace std;

void printarray (int arg[], int length) {
  for (int n=0; n<length; ++n)
    cout << arg[n] << ' ';
  cout << '\n';
}

int main ()
{
  int firstarray[] = {5, 10, 15};
  int secondarray[] = {2, 4, 6, 8, 10};
  printarray (firstarray,3);
  printarray (secondarray,5);
}
```

```
5 10 15
2 4 6 8 10
```

Arrays

- **Range-based for loop**

```
for ( declaration : range ) statement;
```

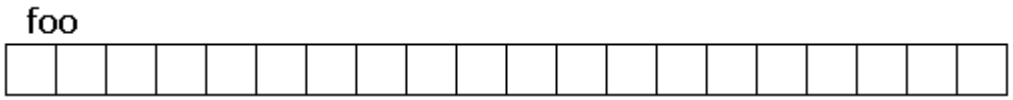
  - This kind of for loop iterates over all the elements in range
  - declaration declares some variable able to take the value of an element in this range.
  - Ranges are sequences of elements, including arrays

language built-in array

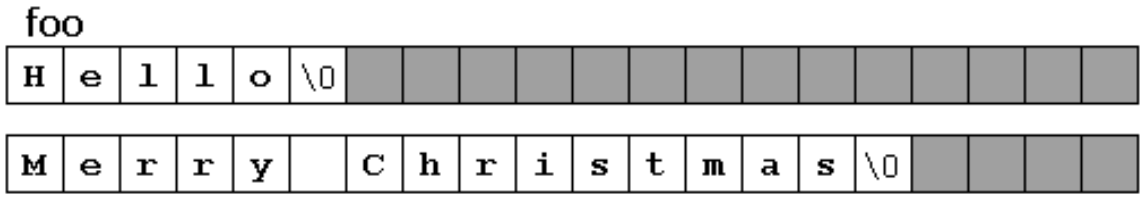
```
#include <iostream>

using namespace std;

int main()
{
  int myarray[3] = {10,20,30};

  for (int i=0; i<3; ++i)
    ++myarray[i];

  for (int elem : myarray)
    cout << elem << '\n';
}
```

Character sequences

- the following array: `char foo [20];`
  - is an array that can store up to 20 elements of type char.



  - this array has a capacity to store sequences of up to 20 characters
  - But this capacity does not need to be fully exhausted: the array can also accommodate shorter sequences
  - at some point in a program, either the sequence "Hello" or the sequence "Merry Christmas" can be stored in foo, since both would fit in a sequence with a capacity for 20 characters.
- the end of strings represented in character sequences is signaled by a special character: the null character, whose can be written as '\0'



Character sequences

- Because arrays of characters are ordinary arrays,
  - they follow the same rules as these.
  - For example, to initialize an array of characters with some predetermined sequence of characters, we can do it just like any other array:

```
char myword[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
```

- But arrays of character elements have another way to be initialized:
  - using string literals directly.
  - specified by enclosing the text between double quotes ("some text")
- Sequences of characters enclosed in " " are literal constants.
  - And their type is, in fact, a null-terminated array of characters.
  - string literals always have a null character ('\0') automatically appended at the end
- Example: In both cases, the array of characters myword is declared with a size of 6 elements of type char

```
1 char myword[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
2 char myword[] = "Hello";
```

Character sequences

- once myword has already been declared as below

```
1 char myword[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
2 char myword[] = "Hello";
```

  - Its sequence of values cannot be changed as above.
  - Only value of each single element can be changed

```
1 myword[0] = 'B';
2 myword[1] = 'y';
3 myword[2] = 'e';
4 myword[3] = '\0';
```

- Example

```
#include <iostream>

using namespace std;

int main ()
{
  char question[] = "What is your name? ";
  char answer [80];
  cout << question;
  cin >> answer;
  cout << "Hello, " << answer << "\n";
  return 0;
}
```

Character sequences

- the string class (excluded)
  - Variables of this type are able to store sequences of characters, such as words or sentences.
  - From library
    - #include <string>

```
#include <iostream>

using namespace std;

int main ()
{
  char question[] = "What is your name? ";
  char answer [80];
  cout << question;
  cin >> answer;
  cout << "Hello, " << answer << "\n";
  return 0;
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main ()
{
  string question = "Where do you live? ";
  string answer;
  cout << question;
  cin >> answer;
  cout << "Hello, " << answer << "!\n";
  return 0;
}
```

```
char myntcs[] = "some text";
string mystring = myntcs;  // convert c-string to string
cout << mystring;          // printed as a library string
cout << mystring.c_str();  // printed as a c-string
```

Character sequences

However, `cin` extraction always considers spaces (whitespaces, tabs, new-line...) as terminating the value being extracted, and thus extracting a string means to always extract a single word, not a phrase or an entire sentence.

To get an entire line from `cin`, there exists a function, called `getline`, that takes the stream (`cin`) as first argument, and the string variable as second. For example:

```
// cin with strings
#include <iostream>
#include <string>
using namespace std;

int main ()
{
  string mystr;
  cout << "What's your name? ";
  getline (cin, mystr);
  cout << "Hello " << mystr << ".\n";
  cout << "What is your favorite team? ";
  getline (cin, mystr);
  cout << "I like " << mystr << " too!\n";
  return 0;
}
```

```
What's your name? Homer Simpson
Hello Homer Simpson.
What is your favorite team? The Isotopes
I like The Isotopes too!
```

Notice how in both calls to `getline`, we used the same string identifier (`mystr`). What the program does in the second call is simply replace the previous content with the new one that is introduced.

Exercises

- Send source code of any 2 of the following programs:
  - 2.1(nested loops): write a program which asks an integer number and writes it in reverse, then asks for another number and writes it in reverse. It does this until it gets 0 as an input, then exits.
    Write it without using functions, you need two nested loops (one inside the other)
  - 2.2(array): write a program which asks for 10 integer numbers, saves them inside an array and then sorts them (small to large) and then shows all the numbers ordered,
    and their sum
    and average
    and max , min
  - 2.3 (recursive functions): write a program which asks two numbers (a and b) and then calculates a^b using recursive function as $a^b = a \times a^{b-1}$
  - 2.4 (characters): ask for two strings (max length 100), then compare them from beginning (character by character) and show if they are the same
    if not the same, then show difference from the point of difference

References

1. http://www.cplusplus.com/doc/tutorial/
2. http://www.tutorialspoint.com/cplusplus/cpp_data_types.htm